Trabalho 1 – Concorrência e Sincronização: O Problema dos Filósofos Jantando

Fundamentos de Processamento Paralelo e Distribuído (FPPD)
Henrique Horch, Felipe Araujo e Eduardo Costa
https://github.com/eduardocsilveira/FPPD_T1

1 Descrição das soluções

Fizemos as quatro versões em Go, uma por arquivo (`v0/main.go` a `v3/main.go`). Todas usam os mesmos parâmetros: 5 filósofos, 20 ciclos e 50 ms tanto para pensar quanto para comer. O filósofo i tem o garfo i à esquerda e o garfo $(i + 1) \bmod 5$ à direita. De uma versão para outra só muda o jeito de pegar os garfos.

A V0 segue a mesma ideia do `filo_seq.go` do Moodle, com a diferença de que usamos 20 ciclos em vez de 5, como o enunciado pede, e 50 ms em vez de 2 s e 3 s; com os tempos originais, cada execução da V0 levaria mais de 8 minutos. A estratégia da V3 é uma das soluções vistas em aula (aula 10): pegar os recursos sempre na mesma ordem.

	Como funciona	**Mecanismo de sincronização**
V0	Dois laços, um dentro do outro: cada filósofo faz seus 20 ciclos de pensar e comer e só depois o próximo começa.	Nenhum. Sem goroutines não tem disputa por garfo.
V1	Uma goroutine por filósofo. Cada um pega o garfo esquerdo, espera 5 ms e pega o direito. O programa imprime quem pegou e quem está esperando.	Um `sync.Mutex` por garfo (`Lock` pega, `Unlock` solta). O `main` espera as goroutines com `sync.WaitGroup`.
V2	Igual à V1, só que antes de pegar os garfos o filósofo trava a mesa inteira, e só destrava depois de soltar os dois.	Os mutexes dos garfos e mais um `sync.Mutex` global, a `mesa`. Na prática é um semáforo binário em volta da refeição toda.
V3	Sem a trava da mesa. Cada filósofo pega primeiro o garfo de número menor. Só muda para o filósofo 4, que passa a pegar o garfo 0 antes do 4.	Um `sync.Mutex` por garfo, sempre travados em ordem crescente de número.

A V1 trava porque os cinco terminam de pensar quase juntos, cada um pega o garfo da esquerda e o da direita já está com o vizinho. A V2 não trava porque só um filósofo por vez chega a pegar garfos. A V3 não trava porque, com a ordem fixa, os cinco nunca ficam cada um com um garfo na mão. A análise detalhada está na Seção 4.

2 Análise comparativa

A V0 levou 10,19 s em média. A V2 caiu para 5,15 s e a V3 para 2,74 s. A V2 é 1,98 vezes mais rápida que a sequencial, mas ainda assim gasta 1,88 vezes o tempo da V3. A diferença aparece melhor na espera: na V2 cada filósofo ficou em média 150,0 ms parado por ciclo, contra 24,8 ms na V3. Na V2 nunca houve mais de 1 filósofo comendo ao mesmo tempo; na V3 chegaram a 2, que é o máximo com cinco garfos. As três versões que terminam fecharam os 20 ciclos de todos os filósofos. A V1 travou em 5 das 5 execuções, logo depois do primeiro pensar.

Em resumo: a V2 resolve o deadlock, mas segura demais. A V3 resolve o mesmo problema deixando dois filósofos comerem juntos, e é isso que explica a diferença de tempo.

3 Resultados experimentais

A Tabela 1 traz a média de 5 execuções de cada versão. A maior espera é o maior valor visto em todas as execuções. Na V0 não medimos espera nem simultaneidade: o programa é sequencial, então a espera é zero e só um come por vez.

Tabela 1: Resultados (tempos em ms, média de 5 execuções).

Versão	Tempo total	Mín–máx	Espera méd.	Maior espera	Máx. juntos	Ciclos F0–F4
V0 (sequencial)	10.185,5	10.182,1–10.188,8	0	0	1	20/20/20/20/20
V2 (trava da mesa)	5.146,0	5.140,6–5.151,4	149,99	204,74	1	20/20/20/20/20
V3 (ordem dos garfos)	2.741,8	2.702,5–2.752,6	24,77	153,11	2	20/20/20/20/20
V1 (mutex por garfo)	deadlock em 5 de 5 execuções, nenhum ciclo concluído (Seção 4.1)					

Tabela 2: V2 e V3 execução por execução (ms).

V2				
Exec.	Total	Espera méd.	Maior esp.	Juntos
1	5.145,6	150,28	203,74	1
2	5.140,6	150,15	204,13	1
3	5.148,0	149,32	204,74	1
4	5.151,4	149,95	204,50	1
5	5.144,5	150,24	203,88	1

V3				
Exec.	Total	Espera méd.	Maior esp.	Juntos
1	2.752,6	26,50	152,64	2
2	2.752,3	23,45	102,57	2
3	2.751,6	24,46	153,11	2
4	2.749,9	25,47	102,59	2
5	2.702,5	23,98	102,44	2

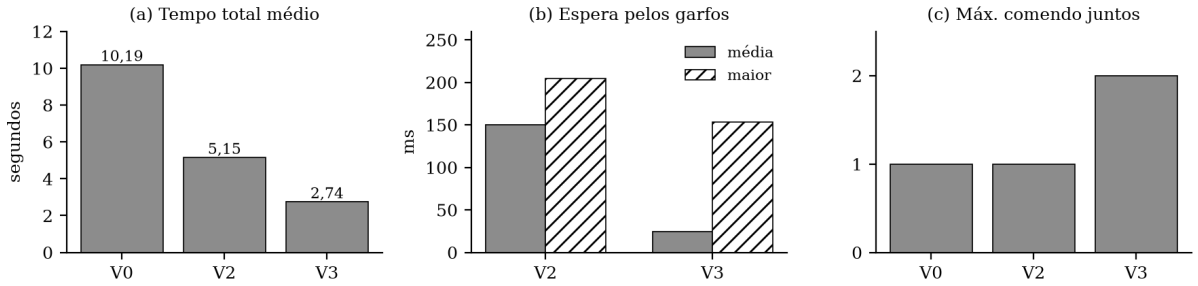


Figura 1: Comparação entre as versões: (a) tempo total médio, (b) espera média e maior espera, (c) máximo de filósofos comendo ao mesmo tempo.



Figura 2: Execuções. À esquerda, a V1 travando (evidência do deadlock); à direita, V0, V2 e V3. Os prints foram tirados no GitHub Codespaces (Linux); a tabela e os gráficos foram medidos no Apple M1 da Seção 5, por isso os tempos não batem exatamente.

4 Conclusão

O que mais pesou no tempo total foi quantos filósofos conseguem comer ao mesmo tempo. Na V0 nada acontece em paralelo e o tempo é a soma de tudo: 100 ciclos de 100 ms, uns 10 s. Na V2 o pensar de um filósofo acontece enquanto outro come, mas as 100 refeições continuam uma atrás da outra (100×50 ms $= 5$ s), e foi exatamente aí que ela ficou: 5.146 ms. Na V3 dois comem juntos, então o melhor tempo possível seria a metade, 2,5 s. Ela ficou 10% acima disso, porque às vezes um filósofo segura o primeiro garfo enquanto espera o segundo e acaba atrapalhando um vizinho que poderia comer.

A espera conta a mesma história. Na V2 dá para prever a média na mão: cinco filósofos dividindo uma vaga, cada um come uma vez a cada 250 ms; tirando 50 ms pensando e 50 ms comendo, sobram 150 ms de fila. Medimos 150,0 ms. Na V3 a média caiu para 24,8 ms, umas 6,1 vezes menos. Os resultados da V2 quase não variaram entre as execuções (5.141 a 5.151 ms), o que mostra que o custo vem da própria solução e não de sorte no escalonamento.

A conclusão principal é que sincronizar mais não deixou o programa melhor. V2 e V3 estão igualmente corretas, mas a V2 proíbe também situações seguras, como F0 e F2 comerem juntos, e paga isso em tempo e em espera.

Uma observação sobre o uso de vários núcleos: nos nossos programas, pensar e comer são `time.Sleep`, que não ocupa o processador. O ganho da V2 e da V3 vem de sobrepor essas esperas, ou seja, de concorrência, e não de executar cálculo em paralelo. Para conferir, rodamos a V2 e a V3 limitando o Go a um núcleo (`GOMAXPROCS=1`) e os tempos quase não mudaram: cerca de 5,4 s e 2,9 s. Se os filósofos fizessem cálculo de verdade, aí sim os 8 núcleos da máquina fariam diferença.

4.1 Análise do deadlock da V1

Filósofo	Possui	Espera	Quem tem
F0	garfo 0	garfo 1	F1
F1	garfo 1	garfo 2	F2
F2	garfo 2	garfo 3	F3
F3	garfo 3	garfo 4	F4
F4	garfo 4	garfo 0	F0

Execução 1 (`resultados/saidas/v1_exec1.txt`).

Cada filósofo segura o garfo da esquerda e espera o da direita, que está com o vizinho. Seguindo a tabela: F0 espera F1, F1 espera F2, F2 espera F3, F3 espera F4 e F4 espera F0. É a espera circular. Ninguém solta o que tem, porque para soltar precisaria comer, e para comer precisaria do garfo do vizinho. O Go percebe que todas as goroutines estão paradas e encerra o programa com a mensagem `fatal error` mostrada na Figura 2. Isso aconteceu em 5 das 5 execuções.

As quatro condições de deadlock estão presentes: **exclusão mútua**, porque um mutex tem um dono só; **segura e espera**, porque o filósofo não larga o garfo esquerdo enquanto espera o direito; **ausência de preempção**, porque só quem deu `Lock` pode dar `Unlock`; e **espera circular**, a volta completa da tabela acima. A pausa de 5 ms entre os dois garfos só deixa o problema mais fácil de ver. Sem ela o deadlock continua possível, mas passa a depender do escalonamento.

4.2 Respostas às questões do enunciado

1. **Como funciona cada solução?**
   A V0 roda os filósofos em fila, um depois do outro. Na V1 cada filósofo vira uma goroutine que pega o garfo esquerdo e depois o direito. A V2 faz a mesma coisa, mas dentro de uma trava que vale para a mesa inteira. A V3 não tem essa trava; nela todos pegam os garfos em ordem crescente de número.

2. **Quais mecanismos de sincronização são utilizados?**
   Exclusão mútua com um mutex por garfo (V1, V2 e V3), uma trava global na V2 e ordenação dos recursos na V3. A V0 não precisa de nenhum.

3. **Como esses mecanismos são implementados na linguagem escolhida?**
   Cada garfo é um `sync.Mutex`, guardado em um vetor `[5]sync.Mutex`. `Lock` bloqueia a goroutine até o garfo ficar livre e `Unlock` devolve. A mesa da V2 é só mais um `sync.Mutex`. Os filósofos são goroutines criadas com `go`, e o `main` espera por elas com `sync.WaitGroup`. Para a ordenação da V3 não precisamos de nada novo: comparamos os dois números e travamos o menor primeiro. As métricas ficam em variáveis globais com um mutex só delas (`mu`), que não entra na disputa pelos garfos.

4. **Por que ocorre deadlock na V1?**
   Porque dá tempo de os cinco pegarem o garfo esquerdo antes de qualquer um pegar o direito. Aí cada um fica esperando um garfo que está com o vizinho, e a espera fecha uma volta completa na mesa (Seção 4.1). Com as quatro condições presentes ao mesmo tempo, ninguém avança.

5. **Como V2 e V3 evitam deadlock?**
   A V2 acaba com o segura e espera: quem vai pegar garfos já está com a mesa, e nessa hora os dois garfos estão livres. A V3 acaba com a espera circular: como o filósofo 4 agora disputa o garfo 0 com o filósofo 0 antes de segurar qualquer coisa, nunca ficam os cinco com um garfo na mão, e sempre sobra um garfo para alguém completar o par. As duas terminaram os 100 ciclos em todas as execuções e passaram pelo detector de corrida do Go (`go run -race`) sem nenhum aviso.

6. **Existe possibilidade de starvation? Justifique.**
   Na V1 travada existe, e para todos, já que ninguém volta a comer. Na V2 e na V3 o algoritmo não garante quem é atendido primeiro, porque o `sync.Mutex` não funciona como uma fila rígida. Em teoria um filósofo poderia ficar para trás se os outros passassem sempre na frente dele. Nos nossos testes isso não aconteceu: todos fecharam os 20 ciclos em todas as execuções, e a maior espera foi de 205 ms na V2 (umas quatro refeições dos outros) e 153 ms na V3 (cerca de 3 refeições de vizinhos). Então é possível em teoria, mas não observamos.

7. **Qual solução apresentou maior tempo total de execução?**
   A V0, com 10.185 ms em média. Entre as concorrentes que terminam, a V2: 5.146 ms contra 2.742 ms da V3.

8. **Qual solução apresentou maior tempo de espera pelos recursos?**
   A V2, com 150,0 ms de espera média e 204,7 ms de maior espera. Na V3 foram 24,8 ms e 153,1 ms. Na V0 não há espera, porque nunca tem outro filósofo ativo.

9. **Quantos filósofos conseguiram comer simultaneamente em cada solução?**
   Na V0, 1. Na V2, 1. Na V3, 2, que é o máximo possível com cinco garfos. Na V1 nenhum chegou a comer, porque o programa travou antes da primeira refeição.

10. **Qual foi o impacto da sincronização excessiva da V2?**
    As refeições viraram uma fila única. O tempo total ficou em 5.146 ms, quase os 5.000 ms das 100 refeições em série, e 1,88 vezes o tempo da V3. Cada filósofo esperou em média 150 ms por ciclo, o triplo do tempo que passa comendo.

11. **A solução com maior sincronização foi necessariamente a de melhor desempenho? Explique com base nas métricas obtidas.**
    Não. A V2 é a que mais restringe os filósofos e foi a pior das versões concorrentes em tudo que medimos: 1,88 vezes o tempo total da V3, 6,1 vezes a espera média e metade dos filósofos comendo juntos. A V3 proíbe só o necessário para não fechar o ciclo de espera; a V2 proíbe também coisas seguras.

12. **Qual é a relação entre o grau de concorrência observado e o tempo total de execução?**
    Quanto mais coisa acontece ao mesmo tempo, menor o tempo total. Na V0 nada se sobrepõe e foram 10,2 s. Na V2 só o pensar se sobrepõe e caiu para 5,1 s. Na V3 as refeições também se sobrepõem (dois comendo) e foi para 2,7 s: dobrar o número de filósofos comendo juntos cortou o tempo quase pela metade. Um detalhe: V0 e V2 têm o mesmo máximo de filósofos comendo (1) e

tempos bem diferentes, então esse número sozinho não mede a concorrência. O pensar ao mesmo tempo também conta.

5 Ambiente de execução

- **Linguagem e versão:** Go 1.27.0 (darwin/arm64)
- **Sistema operacional:** macOS 13.4
- **Processador:** Apple M1
- **Cores disponíveis:** 8
- **Compilação e execução:** `go run ./v0` (o mesmo para `v1`, `v2` e `v3`); o `go run` compila e executa
- **Experimento:** `scripts/rodar.sh` roda 5 vezes cada versão e salva as saídas na pasta `resultados/saidas`
- **Prints:** GitHub Codespaces (Linux), com o mesmo código do repositório; as medições da tabela não foram feitas nele

Link do Git: `https://github.com/eduardocsilveira/FPPD_T1`